

Appendix

Songyao Wang
Tsinghua University
Beijing, China

wangsong23@mails.tsinghua.edu.cn

Furong Chen
Tencent
Shenzhen, China
vinchen@tencent.com

Chaokun Wang
Tsinghua University
Beijing, China

chaokun@tsinghua.edu.cn

Fang Niu
Tsinghua University
Beijing, China
nf21@mails.tsinghua.edu.cn

A CELL MODIFICATION OPERATIONS

The offline construction of a Crosslinked Cell Hierarchy captures graph topology and vector geometry, but it cannot anticipate the query workload. When an evolving workload frequently accesses an Active Cell but selects few of its vectors as final results, the cell exhibits low result utilization. When the workload accesses two adjacent Active Cells together for most queries, the pair exhibits high co-access affinity. Cell Modification Operations adjust these local cell boundaries without changing community membership, Vamana edges, or vector values. These background operations preserve CROSH adaptivity after offline construction and can reduce avoidable read amplification.

For each Active Cell X , SkipCell records its result utilization $u(X)$, defined as the fraction of evaluated vectors in X that enter the final top- k results. For each pair of adjacent Active Cells X and Y , SkipCell records their co-access affinity $a(X, Y)$, defined as the fraction of queries that access both Cells among the queries that access either Cell. After sufficient observations, an Active Cell with persistently low $u(X)$ is split along its internal Vamana topology into two smaller Active Cells. Conversely, adjacent cells with persistently high $a(X, Y)$ are merged when their combined population fits an allowed cell capacity. Minimum sample counts and separate trigger thresholds prevent sparse observations or short workload changes from causing repeated split and merge operations.

The publication procedure follows the nonblocking structural modification design of Bw-Graph. A background thread first materializes the replacement structure as one or more Ghost Cells. The thread completes the Blocks, PQ codes, Cell-4K mappings, hierarchy metadata, and Active Cell Graph connections before making the Ghost Cells visible. It then uses compare-and-swap (CAS) to replace the versioned descriptor of the modified region. Concurrent queries continue to access the stable old version during construction and observe the new version only after successful CAS. The system reclaims the obsolete cells after no active query can reference them.

B COMPLETE TIME COMPLEXITY ANALYSIS OF CWSEARCH

B.1 Computational Model

The analysis counts CPU operations and uses the algorithms stated in Section 4. Let D denote the number of scalar coordinates in each vector. The product quantizer has m_{PQ} subquantizers and K_{PQ} codewords per subquantizer. Computing the query lookup

table compares the query with every codeword across all subspaces, which requires $O(K_{PQ}D)$ scalar operations. After this table has been built, one PQ distance lookup reads one entry for each subquantizer and requires $O(m_{PQ})$ time. One full precision vector distance or one centroid distance requires $O(D)$ time. Fixed width SIMD changes the constant in these bounds but does not change their asymptotic order.

The frontier Q uses a bounded priority queue that supports insertion and removal of either extreme in logarithmic time. The CROSH frontier $\mathcal{F}_i$ uses a minimum heap, while $\hat{\mathcal{P}}_i$ uses a bounded maximum heap whose root supplies τ . The rank operators use comparison sorting, Bottom _{M} uses a bounded maximum heap, and the final TopK uses a maximum heap containing at most k candidates. Membership in $\mathcal{V}$ and access to the community map c require constant time through arrays or direct node identifiers. Grouping the expanded nodes by community can use an ordered map and therefore requires at most $O(L_C \log(L_C + 1))$ time. These choices give worst case CPU bounds for the abstract operations in Algorithms 2 through 4.

Edge Re-allocation produces a graph with maximum node degree $R_{\max}$. Therefore, every node u satisfies

$$|\mathcal{N}_L(u) \cup \mathcal{N}_S(u)| \leq R_{\max}. \quad (1)$$

Let

$$\Delta_H = \max_i \max_{X \in \mathcal{A}_i} \deg_{\mathcal{H}_i}^+(X), \quad N_T = \max_i |\mathcal{T}_i|. \quad (2)$$

Every Active Cell is a nonempty terminal cluster and contains at most $\kappa_{\max}$ vectors by the CROSH construction in Algorithm 1. The budgets L_C , L_A , M , and k are positive integers, while $\omega \in [0, 1]$ is the closeness waterline. Define $Q_A = L_A(R_{\max}^2 + R_{\max} + 2) + \kappa_{\max}$ as an upper bound on the PQ records and local adjacency entries examined by one hybrid CROSH candidate stage.

B.2 Community Skipping

Each iteration of the loop in Algorithm 3 removes one node from Q and adds that node to $\mathcal{E}_q$. The stopping condition therefore permits at most L_C iterations and at most L_C removals from the ordered frontier. The expansion of one node examines at most $R_{\max}$ entries in $\mathcal{N}_L(u) \cup \mathcal{N}_S(u)$. The complete loop consequently examines at most $L_C R_{\max}$ adjacency entries.

An examined entry causes no more than one membership test because $\mathcal{V}$ records every previously encountered node. For an unseen node, PQ scoring costs $O(m_{PQ})$ and updating a frontier of

capacity L_C costs $O(\log(L_C + 1))$. The loop therefore requires

$$O(L_C \log(L_C + 1) + L_C R_{\max}(m_{PQ} + \log(L_C + 1))) \quad (3)$$

time.

The set $\mathcal{E}_q$ contains at most L_C nodes. It therefore represents at most L_C communities. An ordered grouping pass computes every $\sigma_q(C_i)$ in $O(L_C \log(L_C + 1))$ time. Maintaining the best M community scores in a size- M heap costs $O(L_C \log(M + 1))$. Hence,

$$\begin{aligned} T_{CS} = & O(L_C \log(L_C + 1) \\ & + L_C R_{\max}(m_{PQ} + \log(L_C + 1)) \\ & + L_C \log(M + 1)). \end{aligned} \quad (4)$$

B.3 Waterline Search

Consider one root $(i, r_i) \in \mathcal{R}_q$. Because $\mathcal{T}_i$ is a tree, each generated tree node has one parent and enters $\mathcal{F}_i$ at most once. The stopping rule is based on selected leaf population rather than tree depth, so the ordered search can examine every node of $\mathcal{T}_i$ in the worst case. Computing a node bound costs $O(D)$, and its frontier operations cost $O(\log(N_T + 1))$. Hierarchical routing is therefore bounded by

$$O(N_T(D + \log(N_T + 1))). \quad (5)$$

Every selected Active Cell is nonempty. Before the loop selects another leaf, the accumulated population is smaller than L_A . The initial landing set $\mathcal{A}_i^0(q)$ therefore contains at most L_A leaves. Each such leaf has at most Δ_H outgoing neighbors, which gives

$$|\mathcal{Z}_i^q| \leq |\mathcal{A}_i^0(q)|(1 + \Delta_H) \leq L_A(1 + \Delta_H) = Z_A. \quad (6)$$

Scanning the relevant Active Cell Graph edges costs $O(L_A \Delta_H)$. Computing geometric bounds for all correction candidates and sorting the values required by the rank operators cost $O(Z_A D + Z_A \log(Z_A + 1))$. Since $Z_A \geq L_A \Delta_H$, landing correction requires

$$O(Z_A(D + \log(Z_A + 1))) \quad (7)$$

time.

Let $P_i(q) = \sum_{X \in \mathcal{A}_i^q} |X|$ denote the number of vectors in the corrected prefix. If the prefix reaches L_A , its population before the final cell is at most $L_A - 1$, and the final cell adds at most $\kappa_{\max}$ vectors. If the available candidates do not reach L_A , their total population is smaller than L_A . Both cases satisfy

$$P_i(q) \leq L_A + \kappa_{\max} - 1. \quad (8)$$

The Blocks of the sequential set $\mathcal{S}_i^q$ partition its node records, so scanning them examines at most $P_i(q)$ PQ records. In the worst case, no Block is pruned. The graph set $\mathcal{G}_i^q$ contains at most L_A Cells because every Active Cell is nonempty. A two-hop probe examines at most $1 + R_{\max} + R_{\max}^2$ node identifiers, while a one-hop probe examines a subset of this bound. The complete hybrid stage therefore examines fewer than $Q_A = L_A(R_{\max}^2 + R_{\max} + 2) + \kappa_{\max}$ PQ records and local adjacency entries. Computing bounds, scoring candidates, and maintaining the best L_A approximate candidates requires $O(Q_A(m_{PQ} + \log(L_A + 1)))$ time. At most L_A candidates remain after this step. Sorting their physical positions for MERGECONTIGUOUSPAGES costs $O(L_A \log(L_A + 1))$, and their full

precision distances cost $O(L_A D)$. Equations 5–8 therefore give

$$\begin{aligned} T_{CROSH} = & O(N_T(D + \log(N_T + 1)) \\ & + Z_A(D + \log(Z_A + 1)) \\ & + Q_A(m_{PQ} + \log(L_A + 1)) \\ & + L_A(D + \log(L_A + 1))). \end{aligned} \quad (9)$$

B.4 Overall Bound and Scope

Let $m_q = |\mathcal{R}_q|$ be the number of distinct CROSH roots returned by Community Skipping, where $m_q \leq M$. Each Waterline Search invocation returns at most L_A candidates, so their union contains at most $m_q L_A$ candidate occurrences. A maximum heap with capacity k selects the final results in $O(m_q L_A \log(k + 1))$ time. The PQ lookup table is constructed once and shared by both stages. Combining these costs gives the query-specific bound

$$T_{CW} = O(K_{PQ}D + T_{CS} + m_q T_{CROSH} + m_q L_A \log(k + 1)). \quad (10)$$

Using $m_q \leq M$ gives the worst case bound

$$T_{CW} = O(K_{PQ}D + T_{CS} + M T_{CROSH} + M L_A \log(k + 1)). \quad (11)$$

Equation 11 is a CPU operation bound rather than a wall clock latency model. The number and latency of physical reads depend on page placement, cache residency, request merging, and asynchronous I/O overlap. These factors do not compose additively with CPU operations, so the evaluation measures their combined effect instead of assigning a fixed asymptotic latency to each page access.